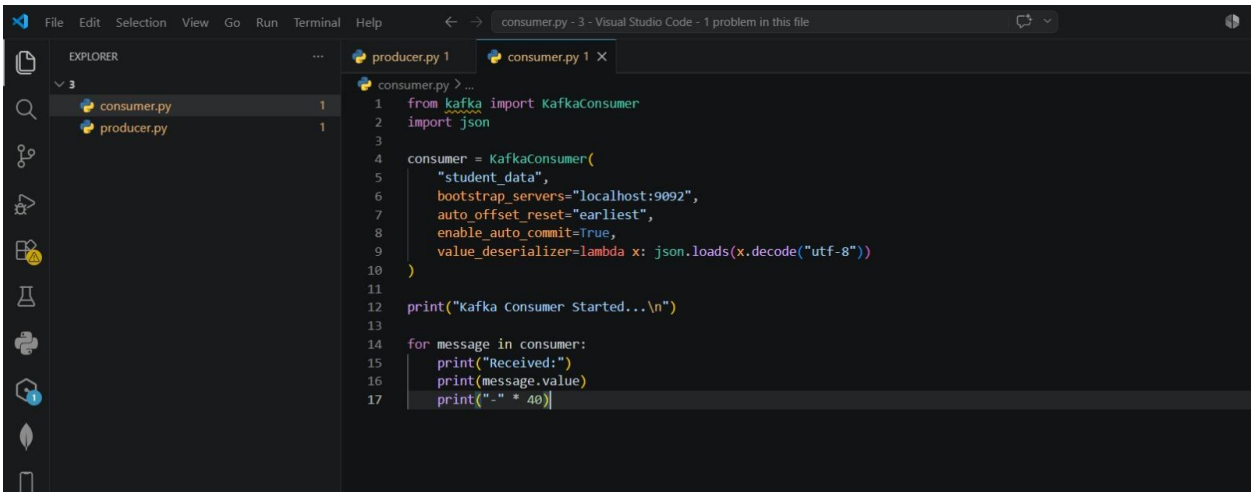
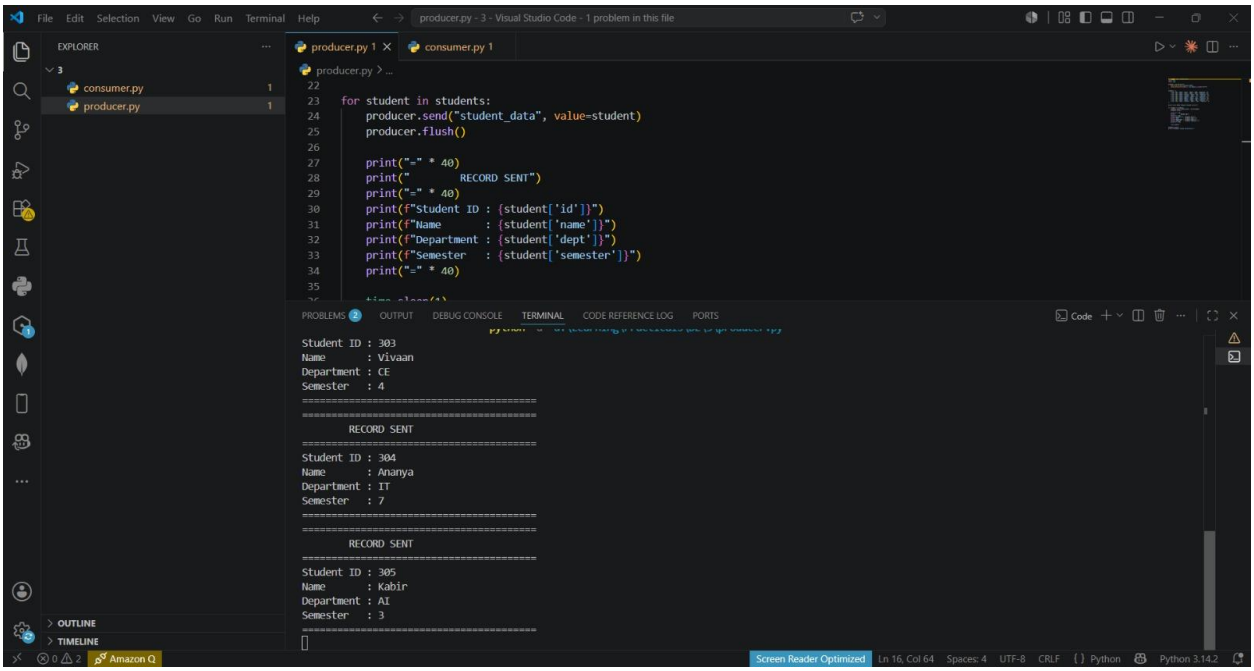
PRACTICAL - 3

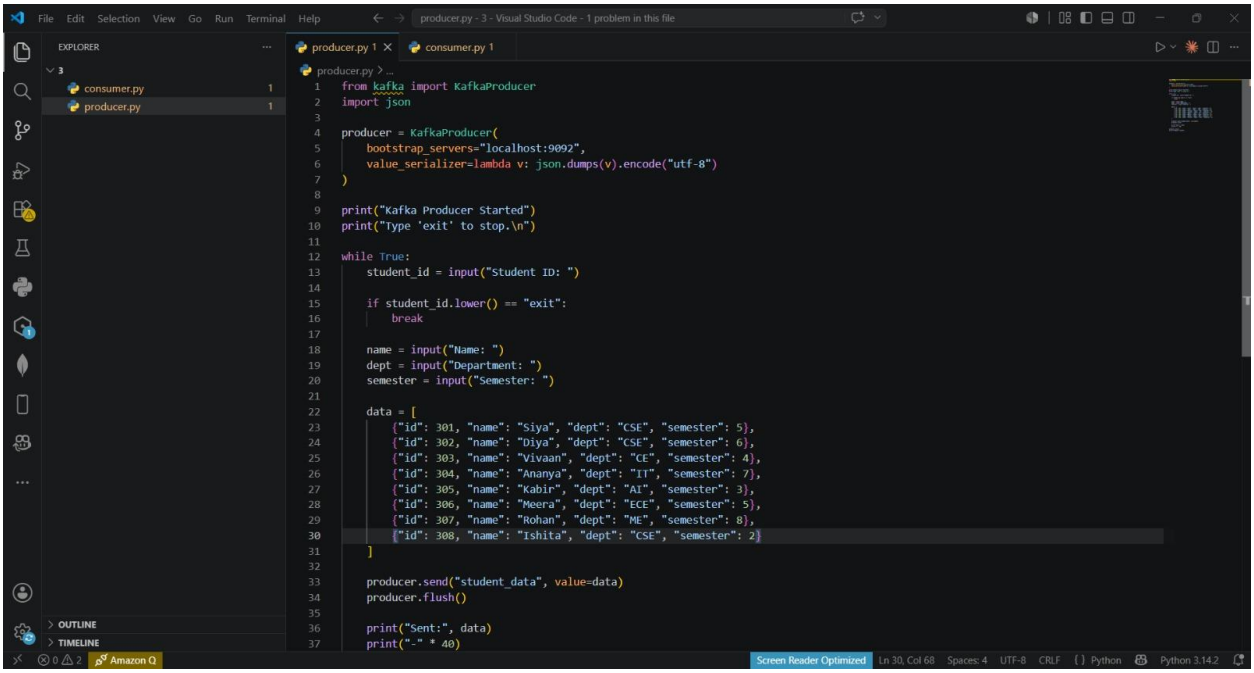
Aim:

Design, spin up, and evaluate two parallel data ingestion pipelines: a periodic timetriggered block batch pipeline and a continuous low-latency real-time event streaming processor.Students must capture telemetry statistics and benchmark performance parameters across both options, measuring data delivery latency, processing throughput spikes, and fault tolerance during simulated infrastructure drops.

Code Screenshot:

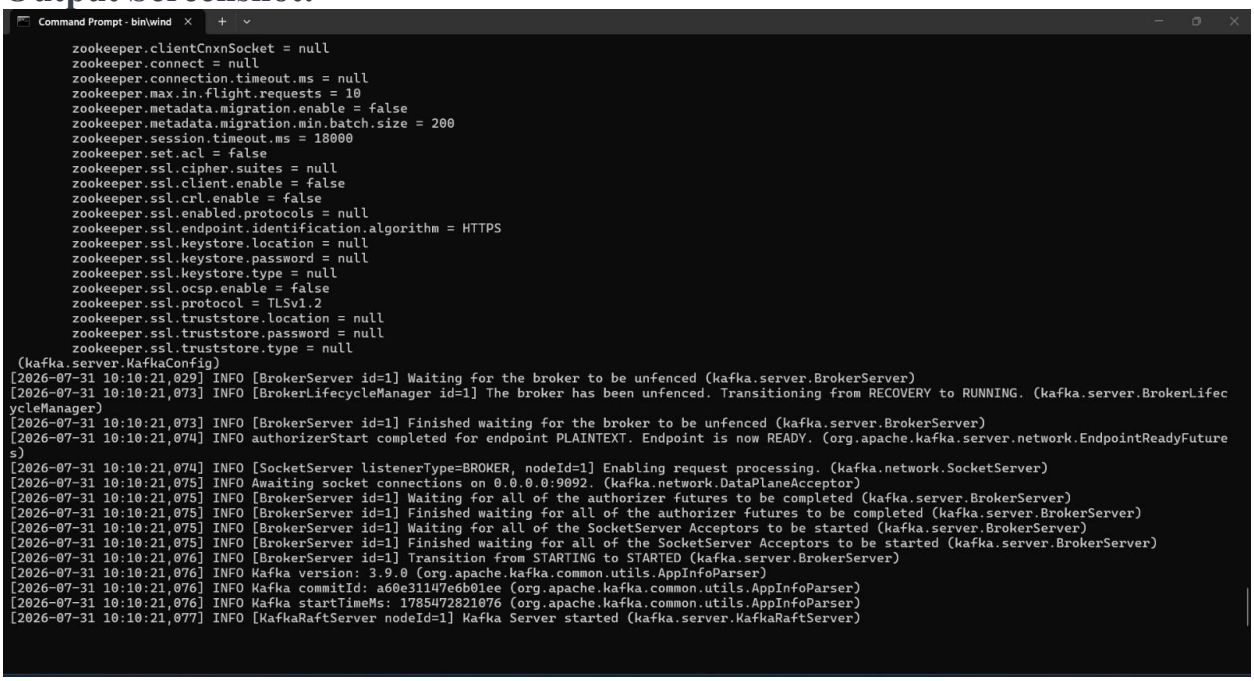




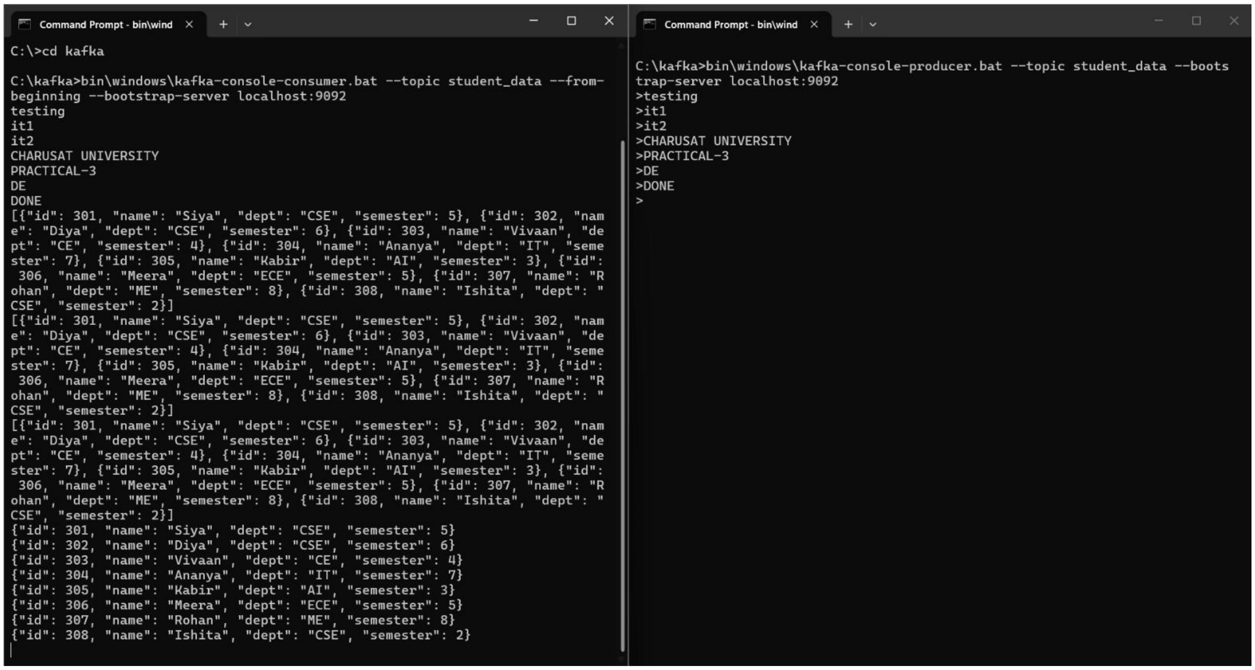


```python
from kafka import KafkaProducer
import json

producer = KafkaProducer(
    bootstrap_servers="localhost:9092",
    value_serializer=lambda v: json.dumps(v).encode("utf-8")
)

print("Kafka Producer Started")
print("Type 'exit' to stop.\n")

while True:
    student_id = input("Student ID: ")

    if student_id.lower() == "exit":
        break

    name = input("Name: ")
    dept = input("Department: ")
    semester = input("Semester: ")

    data = [
        {"id": 301, "name": "Siya", "dept": "CSE", "semester": 5},
        {"id": 302, "name": "Diya", "dept": "CSE", "semester": 6},
        {"id": 303, "name": "Vivaan", "dept": "CE", "semester": 4},
        {"id": 304, "name": "Ananya", "dept": "IT", "semester": 7},
        {"id": 305, "name": "Kabir", "dept": "AI", "semester": 3},
        {"id": 306, "name": "Meera", "dept": "ECE", "semester": 5},
        {"id": 307, "name": "Rohan", "dept": "ME", "semester": 8},
        {"id": 308, "name": "Ishita", "dept": "CSE", "semester": 2}
    ]

    producer.send("student_data", value=data)
    producer.flush()

    print("Sent:", data)
    print("-" * 40)
```

Output Screenshot:



```
        zookeeper.clientCnxnSocket = null
        zookeeper.connect = null
        zookeeper.connection.timeout.ms = null
        zookeeper.max.in.flight.requests = 10
        zookeeper.metadata.migration.enable = false
        zookeeper.metadata.migration.min.batch.size = 200
        zookeeper.session.timeout.ms = 18000
        zookeeper.set.acl = false
        zookeeper.ssl.cipher.suites = null
        zookeeper.ssl.client.enable = false
        zookeeper.ssl.crl.enable = false
        zookeeper.ssl.enabled.protocols = null
        zookeeper.ssl.endpoint.identification.algorithm = HTTPS
        zookeeper.ssl.keystore.location = null
        zookeeper.ssl.keystore.password = null
        zookeeper.ssl.keystore.type = null
        zookeeper.ssl.ocsp.enable = false
        zookeeper.ssl.protocol = TLSv1.2
        zookeeper.ssl.truststore.location = null
        zookeeper.ssl.truststore.password = null
        zookeeper.ssl.truststore.type = null
 (kafka.server.KafkaConfig)
[2026-07-31 10:10:21,029] INFO [BrokerServer id=1] Waiting for the broker to be unfenced (kafka.server.BrokerServer)
[2026-07-31 10:10:21,073] INFO [BrokerLifecycleManager id=1] The broker has been unfenced. Transitioning from RECOVERY to RUNNING. (kafka.server.BrokerLifec
ycleManager)
[2026-07-31 10:10:21,073] INFO [BrokerServer id=1] Finished waiting for the broker to be unfenced (kafka.server.BrokerServer)
[2026-07-31 10:10:21,074] INFO authorizerStart completed for endpoint PLAINTEXT. Endpoint is now READY. (org.apache.kafka.server.network.EndpointReadyFuture
s)
[2026-07-31 10:10:21,074] INFO [SocketServer listenerType=BROKER, nodeId=1] Enabling request processing. (kafka.network.SocketServer)
[2026-07-31 10:10:21,075] INFO Awaiting socket connections on 0.0.0.0:9092. (kafka.network.DataPlaneAcceptor)
[2026-07-31 10:10:21,075] INFO [BrokerServer id=1] Waiting for all of the authorizer futures to be completed (kafka.server.BrokerServer)
[2026-07-31 10:10:21,075] INFO [BrokerServer id=1] Finished waiting for all of the authorizer futures to be completed (kafka.server.BrokerServer)
[2026-07-31 10:10:21,075] INFO [BrokerServer id=1] Waiting for all of the SocketServer Acceptors to be started (kafka.server.BrokerServer)
[2026-07-31 10:10:21,075] INFO [BrokerServer id=1] Finished waiting for all of the SocketServer Acceptors to be started (kafka.server.BrokerServer)
[2026-07-31 10:10:21,076] INFO [BrokerServer id=1] Transition from STARTING to STARTED (kafka.server.BrokerServer)
[2026-07-31 10:10:21,076] INFO Kafka version: 3.9.0 (org.apache.kafka.common.utils.AppInfoParser)
[2026-07-31 10:10:21,076] INFO Kafka commitId: a60e31147e6b01ee (org.apache.kafka.common.utils.AppInfoParser)
[2026-07-31 10:10:21,076] INFO Kafka startTimeMs: 1785472821076 (org.apache.kafka.common.utils.AppInfoParser)
[2026-07-31 10:10:21,077] INFO [KafkaRaftServer nodeId=1] Kafka Server started (kafka.server.KafkaRaftServer)
```



Learning Outcome:

Through this practical, I gained hands-on experience in designing and evaluating both batch and real-time data ingestion pipelines using Kafka/Redpanda, Docker, Python, and Pandas. I learned how to compare latency, throughput, scalability, and fault tolerance between micro-batch and streaming architectures under different workloads and failure scenarios. The implementation enhanced my understanding of message broker operations, consumer group scaling, performance benchmarking, and the factors influencing the selection of batch or streaming solutions for real-world data processing applications.